

Introdução a Classes em Python

Prof. Esp Kelrison Coêlho | CETAM — Linguagem de Programação

Objetivo: aproveitar o Sistema de Biblioteca já desenvolvido para reunir classes, listas, funções e while em um único programa. Não vamos começar um projeto novo.

1. Por que utilizar classes?

Cada livro tem título, autor e situação de empréstimo. Uma classe reúne esses dados e as ações que podemos executar sobre cada livro. As funções e o menu do projeto continuam existindo.

2. Classe x objeto

Classe: modelo de um livro. Objeto: um livro específico criado a partir desse modelo. Uma única classe pode criar vários objetos com dados diferentes.

```
class Livro:
    pass

livro1 = Livro()
livro2 = Livro()
```

class cria o modelo; pass deixa a classe temporariamente vazia. livro1 e livro2 são dois objetos diferentes.

Analogia: a classe é a ficha de cadastro em branco. Cada objeto é uma ficha preenchida para um livro.

3. O que cada parte do projeto fará?

Classe Livro: guarda dados e executa ações do livro. Lista biblioteca: guarda todos os objetos. Funções: organizam cadastro, listagem e empréstimo. while: mantém o menu em execução.

Atributos, __init__ e self

4. Criando livros com seus próprios dados

Atributos são os dados de cada objeto. O método especial __init__ prepara o objeto no momento de sua criação.

```
class Livro:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor
        self.disponivel = True

livro1 = Livro("Dom Casmurro", "Machado de Assis")
livro2 = Livro("O Cortiço", "Aluísio Azevedo")
```

5. Entenda linha por linha

titulo e autor: valores informados ao criar o livro. self: representa o próprio objeto. self.titulo = titulo: armazena o título recebido naquele livro. disponivel = True: todo livro começa disponível.

6. Acessando atributos

```
print(livro1.titulo)      # Dom Casmurro
print(livro2.autor)       # Aluísio Azevedo
print(livro1.disponivel)  # True

livro1.titulo = "Dom Casmurro - 2ª edição"
```

Observe: o ponto (.) permite acessar os atributos. Alterar o título de livro1 não altera livro2.

7. Antes e agora

```
# Antes: lista dentro de lista
biblioteca = [["Dom Casmurro", "Machado de Assis"]]
print(biblioteca[0][0])

# Agora: uma lista pode guardar objetos Livro
biblioteca = [livro1, livro2]
print(biblioteca[0].titulo)
```

A lista continua sendo utilizada. A diferença é que agora cada posição contém um objeto Livro, e não uma lista de dados.

Métodos e lista de objetos

8. Métodos: funções que pertencem à classe

Um método é uma função definida dentro de uma classe. Ele pode acessar e alterar os atributos do objeto por meio de self.

```
class Livro:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor
        self.disponivel = True

    def exibir_dados(self):
        print("Título:", self.titulo)
        print("Autor:", self.autor)
        print("Disponível:", self.disponivel)

    def emprestar(self):
        if self.disponivel:
            self.disponivel = False
            print("Empréstimo realizado!")
        else:
            print("Livro já emprestado!")
```

exibir_dados() mostra as informações. emprestar() aplica a regra de negócio: um livro já emprestado não pode ser emprestado novamente.

9. Criando e percorrendo a biblioteca

```
biblioteca = []

livro1 = Livro("Dom Casmurro", "Machado de Assis")
biblioteca.append(livro1)

for livro in biblioteca:
    livro.exibir_dados()

livro1.emprestar()
```

Ligação com o conteúdo anterior: append() adiciona um objeto à lista; for percorre a lista; o ponto (.) chama o método de cada livro.

Funções e menu com while

10. Organizando as operações do sistema

As funções do programa continuam fora da classe. Elas recebem os dados do usuário, percorrem a lista e chamam os métodos dos objetos.

```
biblioteca = [] # Execute após definir a classe Livro

def cadastrar_livro():
    titulo = input("Título: ")
    autor = input("Autor: ")
    novo_livro = Livro(titulo, autor)
    biblioteca.append(novo_livro)
    print("Livro cadastrado!")

def listar_livros():
    if len(biblioteca) == 0:
        print("Nenhum livro cadastrado.")
    else:
        for livro in biblioteca:
            livro.exibir_dados()
            print("-----")

def emprestar_livro():
    titulo = input("Título para empréstimo: ")
    for livro in biblioteca:
        if livro.titulo == titulo:
            livro.emprestar()
            return
    print("Livro não encontrado.")
```

11. Mantendo o menu funcionando

```
while True:
    print("\n=== BIBLIOTECA ===")
    print("1 - Cadastrar livro")
    print("2 - Listar livros")
    print("3 - Empréstimo livro")
    print("0 - Sair")
    opcao = input("Escolha: ")
    if opcao == "1":
        cadastrar_livro()
    elif opcao == "2":
        listar_livros()
    elif opcao == "3":
        emprestar_livro()
    elif opcao == "0":
        break
    else:
        print("Opção inválida!")
```

Atividade individual: devolução de livros

Missão: refatorar o Sistema de Biblioteca da página anterior. Você já tem cadastro, listagem e empréstimo. Agora implemente a devolução utilizando classe, função, lista e while.

12. Etapa 1 — Um novo método na classe Livro

Crie o método `devolver(self)`. Se o livro estiver emprestado, altere `self.disponivel` para `True` e informe que a devolução foi realizada. Se já estiver disponível, mostre uma mensagem apropriada.

```
class Livro:
    # Mantenha __init__, exibir_dados e emprestar.

    def devolver(self):
        # Complete a condição e as mensagens.
        pass
```

Atenção: o trecho acima mostra apenas o novo método. Acrescente-o à classe `Livro` que você já programou; não substitua os demais métodos.

13. Etapa 2 — Uma função para localizar o livro

Crie a função `devolver_livro()`. Peça o título com `input()`, percorra a lista biblioteca usando `for`, encontre o objeto correspondente e chame `livro.devolver()`. Caso não encontre, informe ao usuário.

14. Etapa 3 — Uma nova opção no while

Acrescente ao menu a opção 4 - Devolver livro e um `elif` para chamar a nova função. Mantenha a opção 0 - Sair.

15. Teste seu programa

Teste 1: cadastre dois livros e liste-os. Teste 2: empreste um deles e tente emprestá-lo novamente. Teste 3: devolva o livro e confira se voltou a ficar disponível. Teste 4: tente devolver um livro já disponível e procure um título inexistente.

Na inspeção individual: identifique no seu código a classe, um objeto, dois atributos, um método, a lista de objetos, uma função e o laço `while`. Explique a diferença entre `emprestar()` e `emprestar_livro()`.